

ISTANBUL TECHNICAL
UNIVERSITY

FACULTY OF AERONAUTICS AND
ASTRONAUTICS

UZF 421E - ATTITUDE DETERMINATION AND
CONTROL

HOMEWORK REPORT #5

B-Dot Control Simulation with Magnetorquers

Ahmet Fevzi Akargöl
110200170

Contents

1	Introduction	2
2	System Definition and Models	2
2.1	Key Parameters and Equations	2
2.2	Sensor Error Models	2
3	Control System and Simulation Methodology	3
3.1	B-Dot Control Law	3
3.2	Simulation Setup	3
4	Simulation Results and Analysis	3
4.1	Case 1: Mag (Bias+Noise), Gyro (Bias+Noise)	4
4.2	Case 2: Mag (Bias+Noise), Gyro (Noise Only)	6
4.3	Case 3: Mag (Noise Only), Gyro (Bias+Noise)	8
4.4	Case 4: Mag (Noise Only), Gyro (Noise Only)	10
5	Comparative Analysis and Discussion	12
6	Conclusion	13
A	Appendix: MATLAB Code	15

1 Introduction

This report details the simulation of a satellite's attitude control using magnetorquers and the B-dot control law, as part of UZB 421E Homework 5. The primary focus is to analyze the control system's performance under different sensor error models, specifically noise and bias for the magnetometer and gyroscope. Simulations were performed in MATLAB using Euler integration for the satellite's dynamic and kinematic equations, integrating knowledge from previous assignments.

2 System Definition and Models

2.1 Key Parameters and Equations

The satellite's attitude is described by 3-2-1 Euler angles (ψ, θ, ϕ) , and its rotational motion by angular velocities $(\omega_1, \omega_2, \omega_3)$ in the body frame (B) relative to the inertial frame (N).

- **Moments of Inertia:** $I_1 = 2.1 \times 10^{-3}$, $I_2 = 2.0 \times 10^{-3}$, $I_3 = 1.9 \times 10^{-3}$ kg · m². Minimum inertia $I_{min} = I_3$.
- **Disturbance Torques (L_D):** $L_{Di} = 3.6 \times 10^{-10}$ N · m for $i = 1, 2, 3$.
- **Kinematic Equations:** Standard 3-2-1 Euler angle rates formulation (Homework 5, Eq. 1).
- **Dynamic Equations:** Euler's rotational equations of motion including disturbance and control torques L_C (Homework 5, Eq. 2-4).

2.2 Sensor Error Models

Sensor measurements are based on models from previous homeworks, with $n = 0$ for student ID dependent parameters.

- **Magnetometer (from HW3):**
 - B_N : Hypothetical inertial magnetic field model (sinusoidal, $T_{orbit} = 6000s$).
 - Bias $b_{mag} = [500; 900; 1200]^T$ nT. Noise $\sigma_{mag} = 100$ nT.
 - $B_b^{Case\ 1} = C_{BN}B_N + b_{mag} + v_{mag}$ (Bias + Noise)
 - $B_b^{Case\ 2} = C_{BN}B_N + v_{mag}$ (Noise Only)
 - $B_b^{Case\ 3} = C_{BN}B_N$ (True body field)
- **Gyroscope (from HW4):**

- Bias b_{rg} : Random walk model; initial $b_{rg,0} = [5; 1; 1] \times 10^{-3}$ deg/s; noise $\sigma_{rgb} = 1 \times 10^{-3}$ deg/s/ $\sqrt{\text{Hz}}$.
- Noise v_{rg} : $\sigma_{rg} = 1 \times 10^{-2}$ deg/s.
- $\tilde{\omega}_{BN}^{Case\ 1} = \omega_{BN}^{true} + b_{rg} + v_{rg}$ (Bias + Noise)
- $\tilde{\omega}_{BN}^{Case\ 2} = \omega_{BN}^{true} + v_{rg}$ (Noise Only)

3 Control System and Simulation Methodology

3.1 B-Dot Control Law

- Magnetic Dipole Moment: $\mathbf{m}_{generated} = -\frac{k_B}{\|\mathbf{B}_b^{measured}\|}(\hat{\mathbf{b}}^{measured} \times \tilde{\omega}_{BN})$
- Applied Torque: $\mathbf{L}_{applied} = \mathbf{m}_{generated} \times \mathbf{B}_b^{Case\ 3}$
- Commanded Torque (for comparison): $\mathbf{L}_{commanded} = -k_B(\mathbf{I}_{3 \times 3} - \hat{\mathbf{b}}^{measured}(\hat{\mathbf{b}}^{measured})^T)\tilde{\omega}_{BN}$
- Controller Gain (k_B): $k_B = \frac{4\pi}{T_{orbit}}(1 + \sin \xi_m)I_{min}$. With $T_{orbit} = 6000$ s, $\xi_m = 80^\circ$, and $I_{min} = 1.9 \times 10^{-3}$ kg · m², $k_B \approx 7.898246 \times 10^{-6}$.

3.2 Simulation Setup

- Initial Conditions for $n = 0$: $\psi_0 = -0.005$, $\theta_0 = 0.01$, $\phi_0 = -0.01$ rad; $\omega_{10} = -0.002$, $\omega_{20} = 0.003$, $\omega_{30} = -0.004$ rad/s.
- Simulation: $N_{samples} = 6000$, $\Delta t = 1$ s. Numerical integration via Euler method. Direction Cosine Matrix (DCM, C_{BN}) updated each step.
- Investigated Cases (k):
 1. Mag Case 1 (Bias+Noise), Gyro Case 1 (Bias+Noise)
 2. Mag Case 1 (Bias+Noise), Gyro Case 2 (Noise Only)
 3. Mag Case 2 (Noise Only), Gyro Case 1 (Bias+Noise)
 4. Mag Case 2 (Noise Only), Gyro Case 2 (Noise Only)

4 Simulation Results and Analysis

This section presents the analysis of simulation results for the four cases, based on the provided numerical outputs. Graphical representations are referred to by their figure names.

4.1 Case 1: Mag (Bias+Noise), Gyro (Bias+Noise)

This worst-case scenario shows significant degradation in control performance.

- $L_{applied}$ (Fig. 1): Initially peaks around 1.3×10^{-8} N.m, then fluctuates with a mean near 10^{-9} - 10^{-10} N.m but high standard deviation ($\sim 3 - 4 \times 10^{-9}$ N.m).
- $||L_{comm} - L_{app}||$ (Fig. 2): Mean of 1.66×10^{-10} N.m.
- **Euler Angles** (Fig. 3): Show severe drift; e.g., Yaw (ψ) reaches -603.4° at 6000s. Pitch (θ) ends at -55.2° , Roll (ϕ) at 67.3° . No stabilization is achieved.
- **Angular Velocities** (Fig. 4): Initial damping is followed by persistent non-zero rates. At 6000s, $\omega \approx [-0.114, -0.077, -0.061]^T$ deg/s. Effective damping fails.

Assessment: Combined biases and noise prevent stabilization. Gyro bias (random walk) is particularly detrimental.

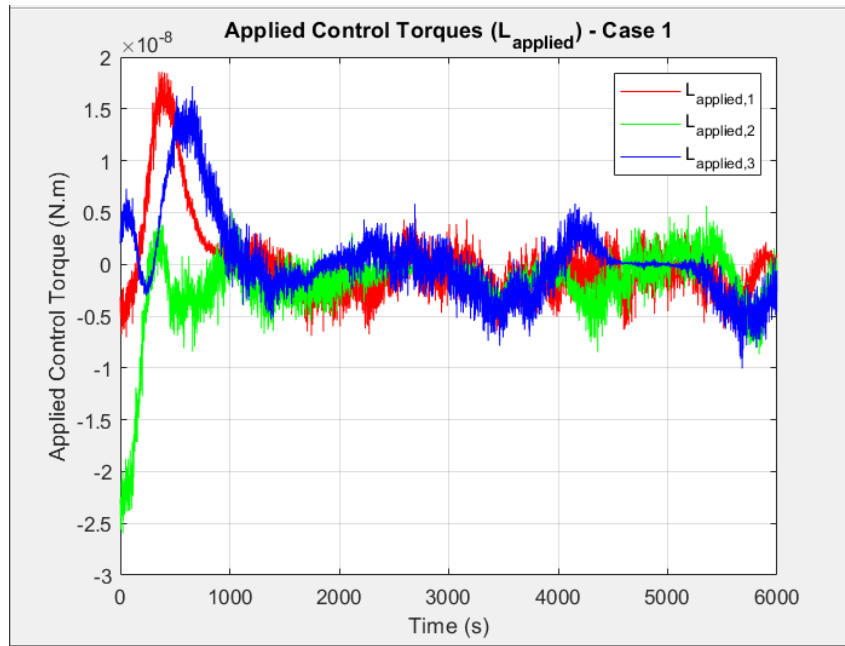


Figure 1: Applied Torques - Case 1

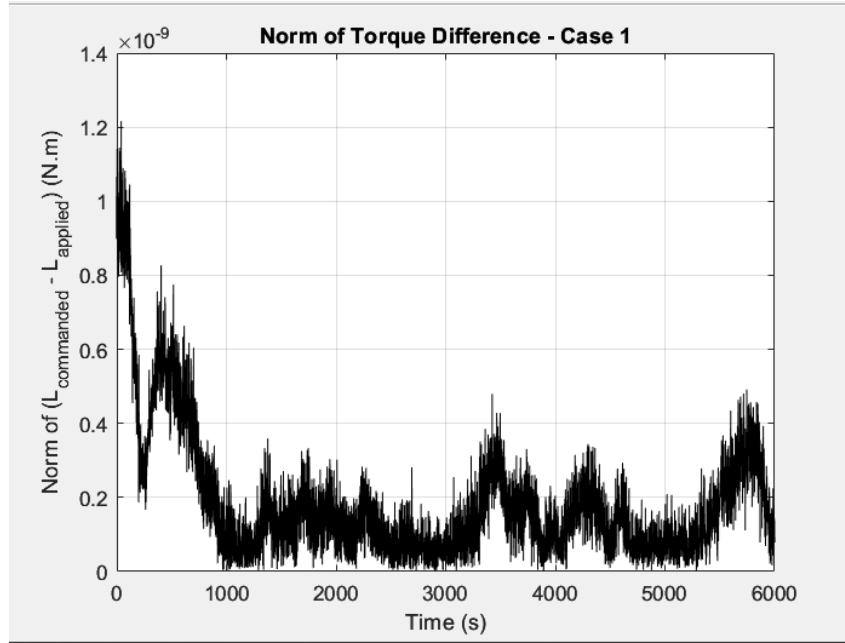


Figure 2: Torque Difference Norm - Case 1

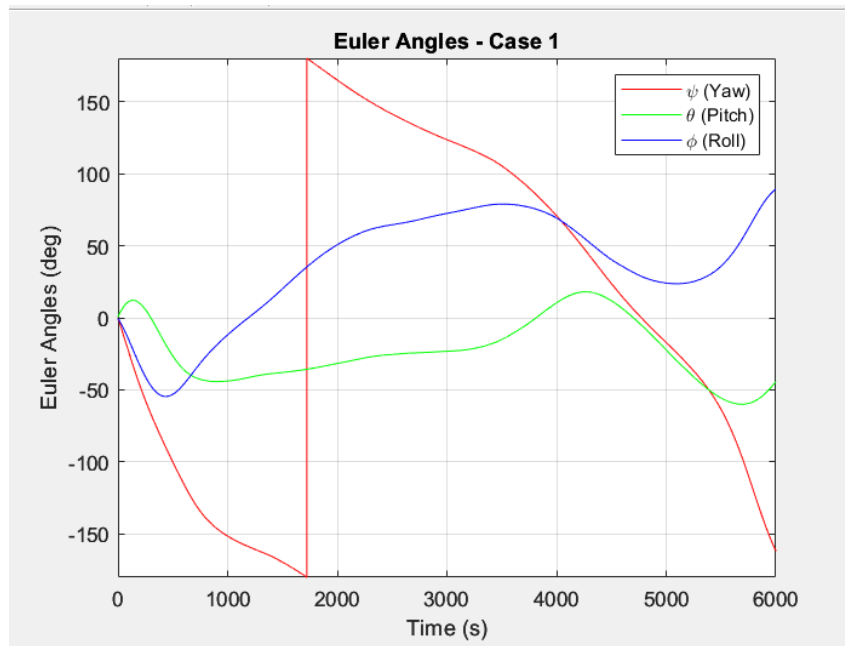


Figure 3: Euler Angles - Case 1

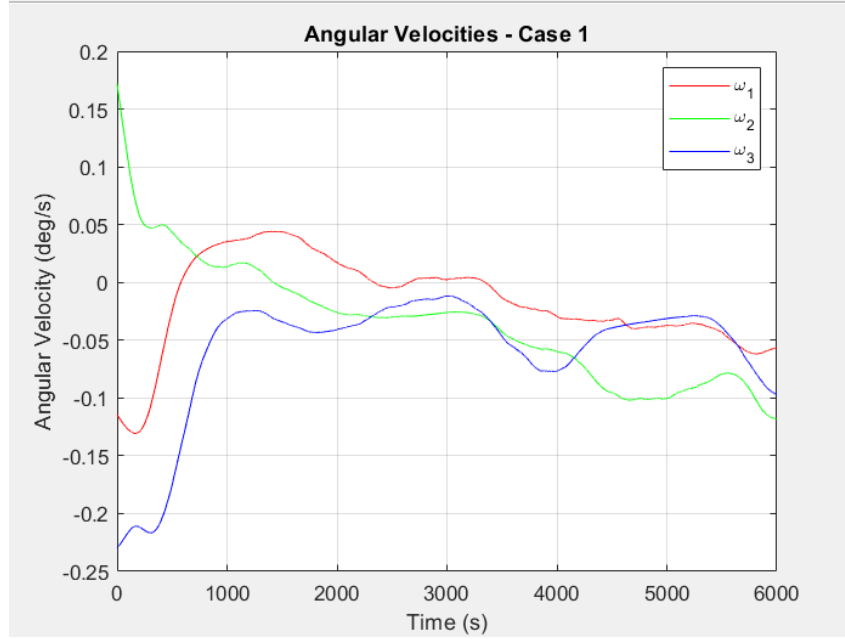


Figure 4: Angular Velocities - Case 1

4.2 Case 2: Mag (Bias+Noise), Gyro (Noise Only)

Removing gyro bias improves performance, especially in rate damping.

- $L_{applied}$ (Fig. 5): Similar magnitude and variability to Case 1. Mean applied torques near 10^{-9} - 10^{-10} N.m.
- $\|L_{comm} - L_{app}\|$ (Fig. 6): Mean of 1.39×10^{-10} N.m, slightly better than Case 1.
- **Euler Angles** (Fig. 7): Yaw (ψ) still drifts significantly (-159.4° at 6000s). Pitch ($\theta \approx 10.5^\circ$) and Roll ($\phi \approx 44.0^\circ$) show better convergence towards quasi-steady values compared to Case 1.
- **Angular Velocities** (Fig. 8): Effectively damped to near-zero values (e.g., at 6000s, $\omega \approx [-0.005, 0.025, 0.016]^T$ deg/s).

Assessment: Gyro bias removal greatly aids rate damping. Magnetometer bias still causes attitude drift, particularly in Yaw.

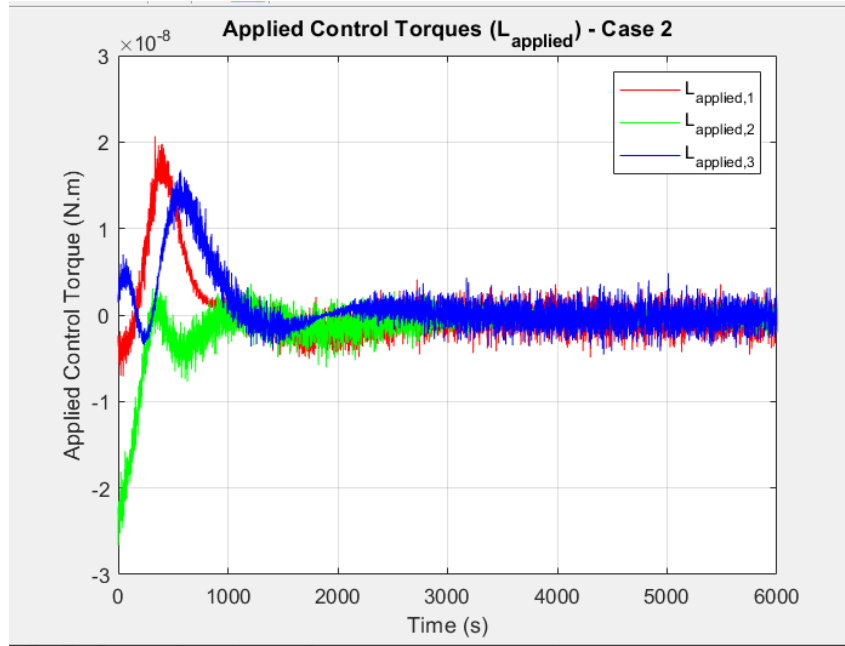


Figure 5: Applied Torques - Case 2

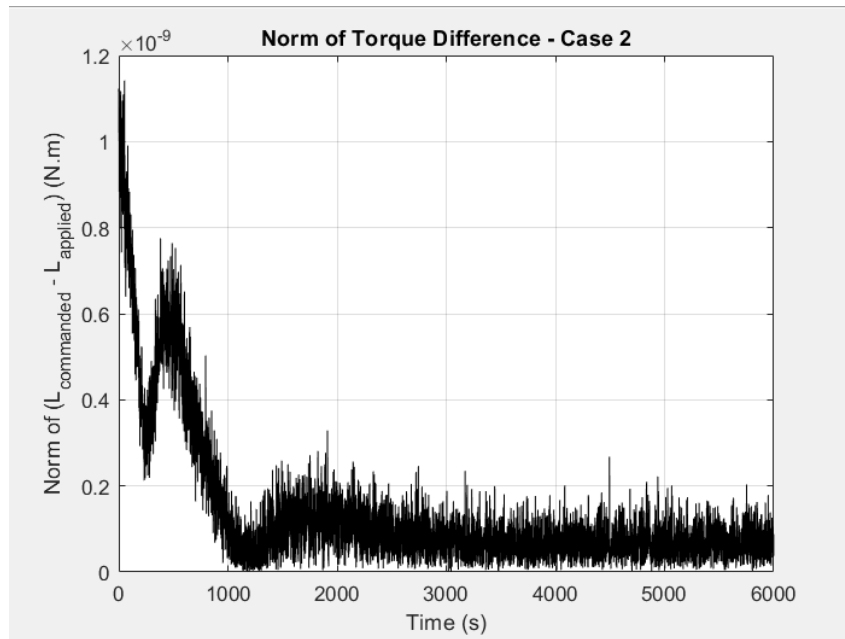


Figure 6: Torque Difference Norm - Case 2

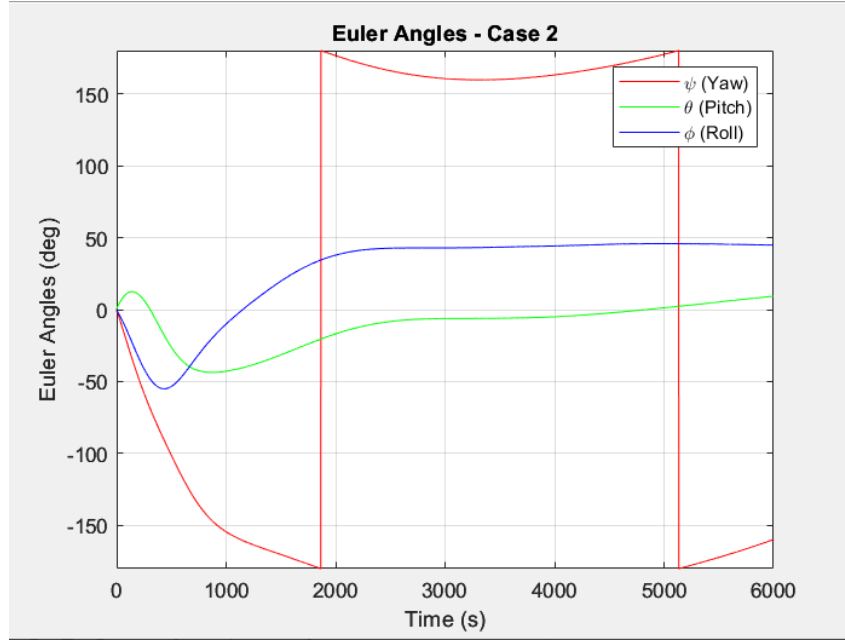


Figure 7: Euler Angles - Case 2

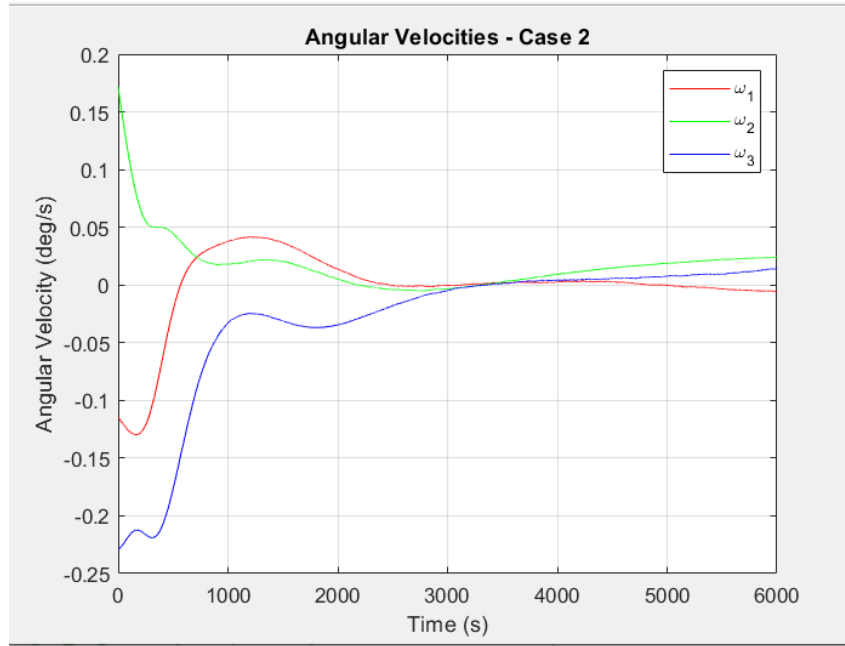


Figure 8: Angular Velocities - Case 2

4.3 Case 3: Mag (Noise Only), Gyro (Bias+Noise)

Removing magnetometer bias while keeping gyro bias does not yield good attitude control.

- $L_{applied}$ (Fig. 9): Similar torque magnitudes and variability as Case 1.
- $\|L_{comm} - L_{app}\|$ (Fig. 10): Significantly lower mean of 1.90×10^{-11} N.m. This indicates better agreement between commanded and applied torques due to a more accurate magnetometer reading of B_b direction.

- **Euler Angles** (Fig. 11): Severe drift persists; Yaw (ψ) reaches -564.6° at 6000s. Pitch ($\theta \approx -41.7^\circ$) and Roll ($\phi \approx 63.6^\circ$) also show large, unstable values.
- **Angular Velocities** (Fig. 12): Not effectively damped, similar to Case 1. At 6000s, $\omega \approx [-0.080, -0.056, -0.002]^T$ deg/s.

Assessment: Gyroscope bias remains the dominant factor for poor attitude and rate control, even with a bias-free magnetometer.

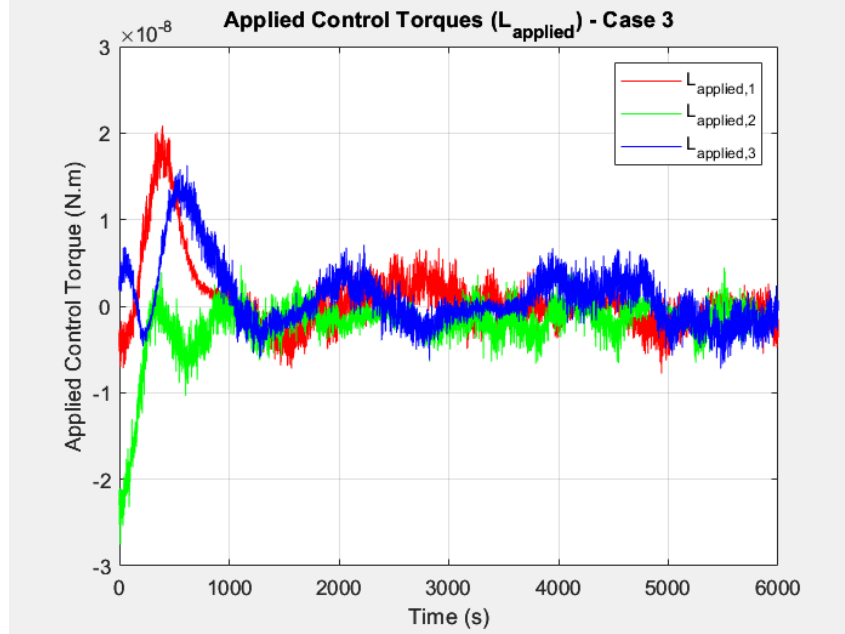


Figure 9: Applied Torques - Case 3

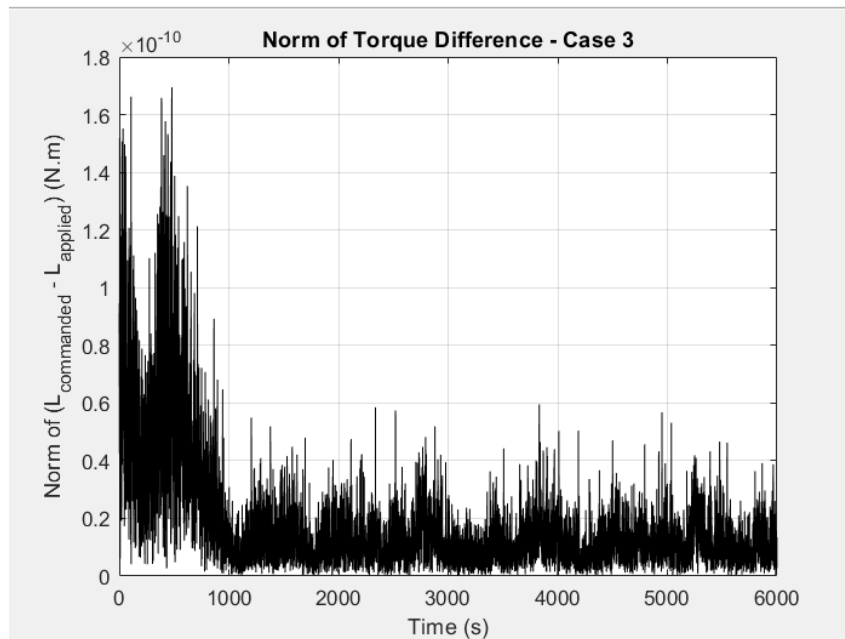


Figure 10: Torque Difference Norm - Case 3

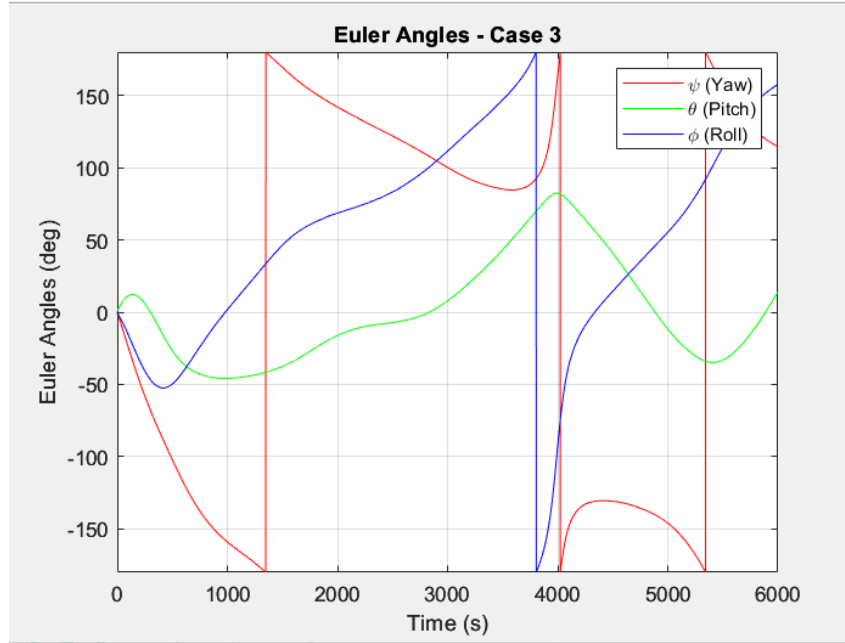


Figure 11: Euler Angles - Case 3

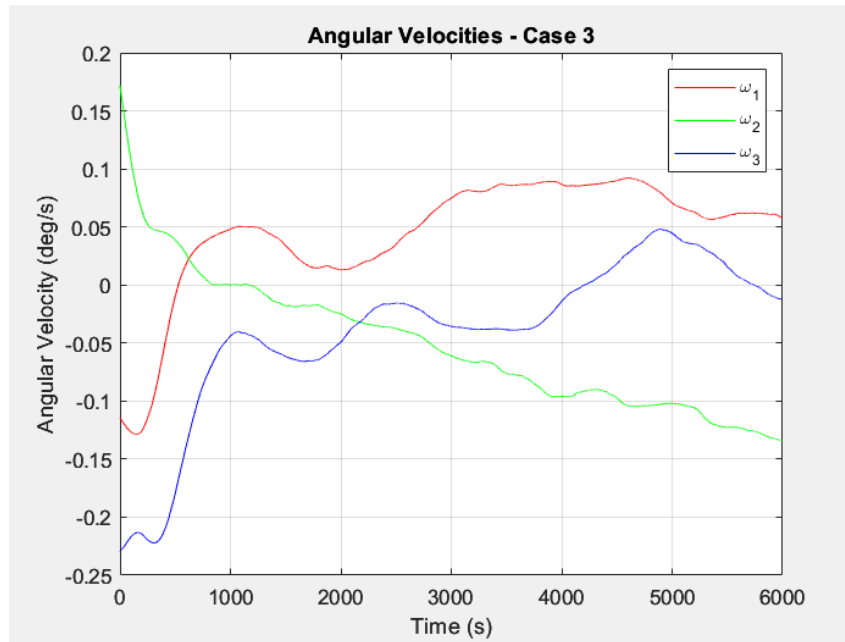


Figure 12: Angular Velocities - Case 3

4.4 Case 4: Mag (Noise Only), Gyro (Noise Only)

This scenario, with biases removed from both sensors, yields the best performance.

- $L_{applied}$ (Fig. 13): Torque magnitudes and variability are similar to other cases initially, driven by the need to damp rates and correct attitude based on noisy data.
- $\|L_{comm} - L_{app}\|$ (Fig. 14): Lowest mean norm of 1.37×10^{-11} N.m, similar to Case 3.

- **Euler Angles** (Fig. 15): Best stabilization. Yaw (ψ) drift is reduced to -157.1° at 6000s (comparable to Case 2). Pitch ($\theta \approx 16.1^\circ$) and Roll ($\phi \approx 46.0^\circ$) converge to relatively steady values.
- **Angular Velocities** (Fig. 16): Most effectively damped. At 6000s, $\omega \approx [-0.005, 0.027, 0.016]^T$ deg/s.

Assessment: The B-dot controller effectively damps angular rates and achieves reasonable attitude stability when sensor biases are absent, tolerating measurement noise.

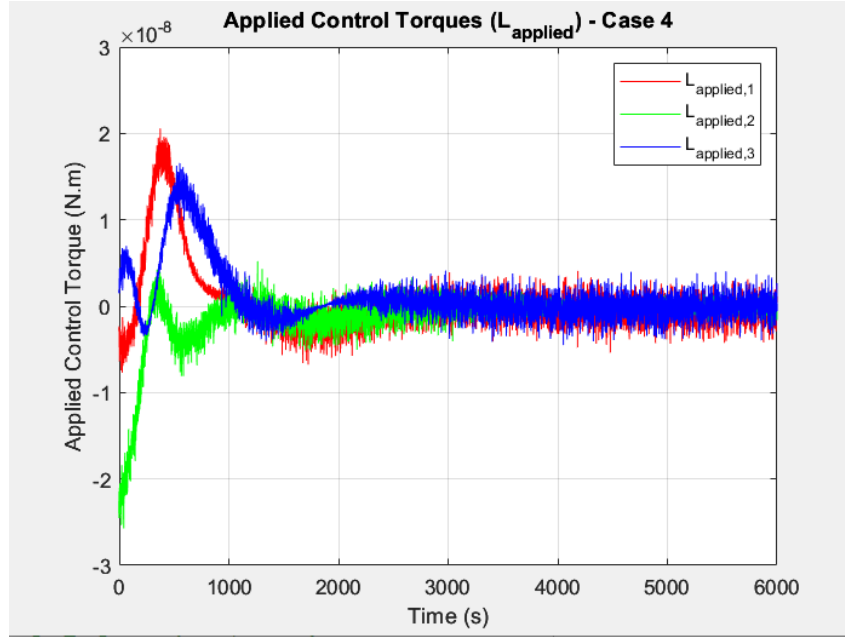


Figure 13: Applied Torques - Case 4

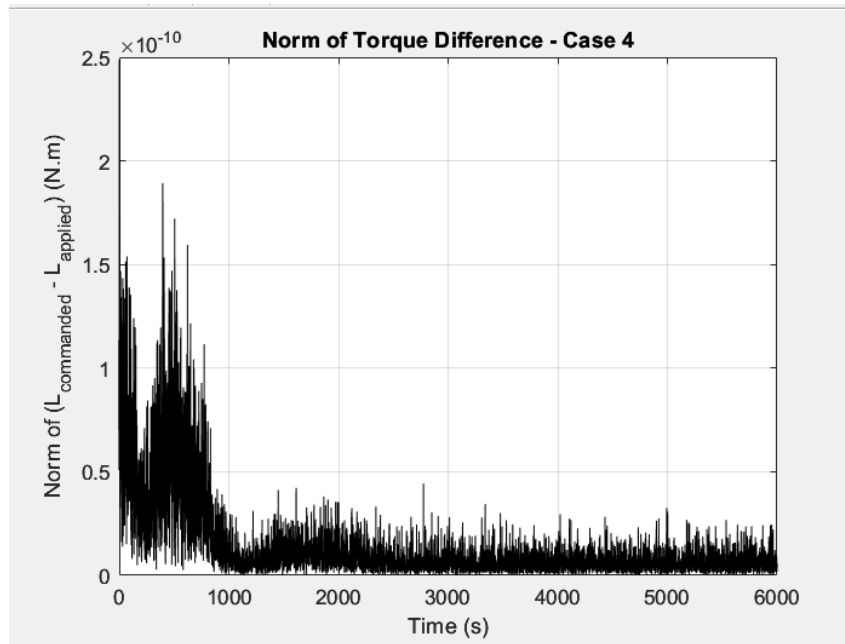


Figure 14: Torque Difference Norm - Case 4

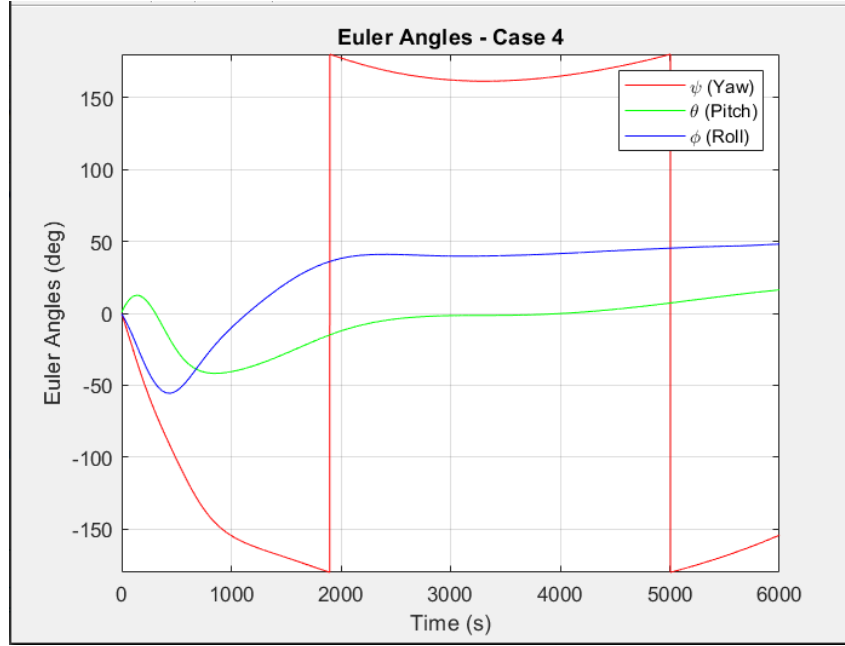


Figure 15: Euler Angles - Case 4

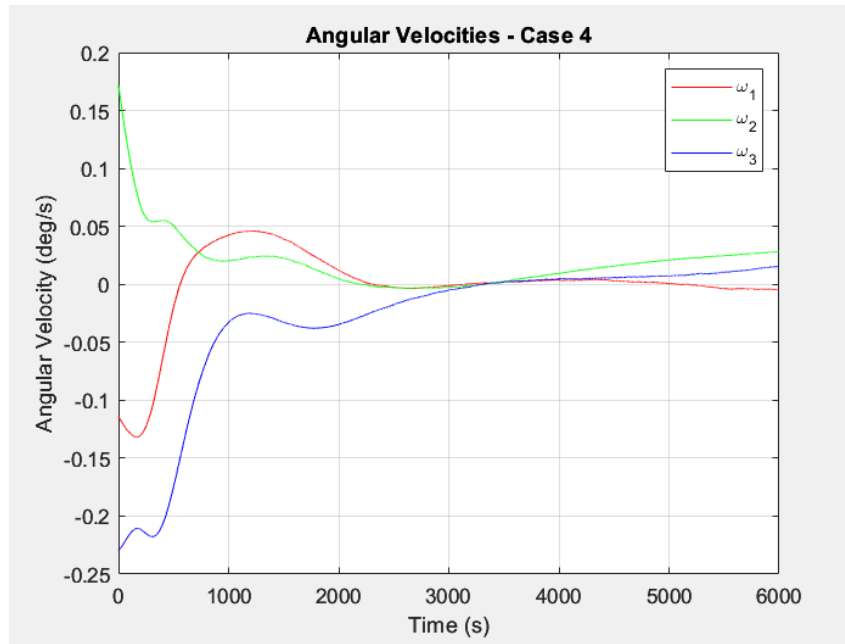


Figure 16: Angular Velocities - Case 4

5 Comparative Analysis and Discussion

Comparing the four cases reveals distinct impacts of sensor errors:

- **Overall Performance:** Case 4 (noise only in both sensors) provided the best performance, characterized by effective angular rate damping and the most stable

attitude. Case 1 (bias and noise in both sensors) exhibited the worst performance, with significant attitude drift and poor rate damping.

- **Impact of Gyroscope Bias:** The presence of gyroscope bias (Cases 1 and 3) was found to be highly detrimental, leading to large attitude drifts and inability to nullify angular rates. The time-varying nature of the random walk bias exacerbates this issue.
- **Impact of Magnetometer Bias:** Magnetometer bias (Cases 1 and 2) also contributed to attitude drift (notably in Yaw for Case 2) and increased the discrepancy between commanded and applied torques. However, its impact on overall stability and rate damping was less severe than gyroscope bias.
- **Impact of Noise:** Sensor noise (all cases) introduced fluctuations but was generally manageable by the B-dot controller, especially when biases were absent (Case 4).
- **Torque Difference:** The norm of $\|L_{commanded} - L_{applied}\|$ was lowest in Cases 3 and 4, where the magnetometer was bias-free. This suggests that a more accurate magnetometer perception of the magnetic field direction (even if noisy) improves the alignment of the control effort.
- **Comparison with Homework 1 (Uncontrolled):** In contrast to the uncontrolled scenario of Homework 1 where attitude would freely drift and angular velocities would persist, the B-dot controller (even with sensor errors) actively attempts to manage the satellite's rotation. In Cases 2 and 4, it successfully damps angular velocities, which is a primary function of B-dot. However, achieving precise attitude pointing is challenging for this simple controller, especially with flawed sensor data. This assignment builds upon previous homeworks by integrating dynamics, kinematics, sensor models, and a control system into a more comprehensive simulation.

6 Conclusion

This study successfully simulated a B-dot attitude control system using magnetorquers, analyzing its performance under different sensor error conditions. The simulations confirm that the B-dot algorithm can effectively damp angular velocities, but its attitude stabilization capability is highly dependent on sensor quality.

The primary findings indicate that sensor biases, particularly time-varying gyroscope bias, are the most critical factors degrading control performance, leading to significant attitude drift and ineffective rate damping. Measurement noise introduces fluctuations but is more tolerable. The best performance was achieved when both sensors were modeled without bias (Case 4), demonstrating successful rate damping and reasonable attitude

stability. Conversely, Case 1, with biases in both sensors, resulted in highly unstable behavior.

This highlights the crucial need for accurate sensor calibration and potentially bias estimation techniques in practical satellite attitude control systems to ensure the effectiveness of even simple control laws like B-dot. While B-dot is suitable for initial detumbling and coarse stabilization, precise pointing requires more advanced control strategies and higher-fidelity sensor information.

A Appendix: MATLAB Code

```

1 %% UZH 421E - Attitude Determination and Control - Homework 5
2 % Ahmet Fevzi Akargöl
3 % Student ID: 110200170
4 % B-Dot Control with Magnetorquers Simulation
5 clc; clear; close all;
6 %% Student Number and Simulation Parameters
7 n_student = 0; % renci numaras n n son hanesi (Sizin i in 0)
8 N_samples = 6000; % rnek say s
9 delta_t = 1; % rnekleme zaman (s)
10 time_vec = (0:N_samples)*delta_t; % Zaman vekt r
11 rad2deg_val = 180/pi;
12 deg2rad_val = pi/180;
13 %% Initial Conditions (from HW1, n=0)
14 % Initial attitude angles (rad)
15 psi_0 = -0.005 - 0.002*n_student; % Yaw
16 theta_0 = 0.01 - 0.002*n_student; % Pitch
17 phi_0 = -0.01 - 0.002*n_student; % Roll
18 euler_initial = [psi_0; theta_0; phi_0]; % S tun vekt r olarak
19 % Initial angular velocities (rad/s)
20 w1_0 = -0.002 - 0.0001*n_student;
21 w2_0 = 0.003 - 0.0001*n_student;
22 w3_0 = -0.004 - 0.0001*n_student;
23 omega_initial = [w1_0; w2_0; w3_0]; % S tun vekt r olarak
24 %% Satellite Parameters (from HW1)
25 % Mass moments of inertia (kg.m^2)
26 I1 = 2.1e-3;
27 I2 = 2.0e-3;
28 I3 = 1.9e-3;
29 I_satellite = diag([I1, I2, I3]); % Eylemsizlik matrisi
30 I_min = I3; % Minimum eylemsizlik momenti (kg.m^2)
31 % External disturbance torques (N.m) - Assuming constant
32 L_D = [3.6e-10; 3.6e-10; 3.6e-10]; % S tun vekt r olarak
33 %% Orbit Parameters (from HW2 & HW3)
34 T_orbit = 6000; % Y r nge periyodu (s) (HW3'teki varsay m)
35 xi_m = 80 * deg2rad_val; % Y r nge e ikli i (rad) (HW2'den, n=0
    i in 80 deg)
36 %% B-Dot Controller Gain (k_B)
37 k_B = (4*pi/T_orbit) * (1 + sin(xi_m)) * I_min;
38 fprintf('Calculated B-Dot Gain (k_B): %e\n', k_B);
39 %% Sensor Error Models (from HW3 & HW4, n=0)
40 % --- Magnetometer Errors (from HW3) ---
41 % Bias vector (nT)
42 b_mag_nT = [500; 900; 1200] + 100*n_student; % nT
43 % Noise standard deviation (nT)
44 sigma_mag_nT = 100; % nT

```



```

45 % --- Gyroscope Errors (from HW4) ---
46 % Initial bias (deg/s)
47 b_rg_0_deg_s = [5; 1+n_student; 1-n_student] * 1e-3; % deg/s
48 b_rg_0_rad_s = b_rg_0_deg_s * deg2rad_val; % rad/s
49 % Measurement noise standard deviation (deg/s)
50 sigma_rg_deg_s = 1e-2; % deg/s
51 sigma_rg_rad_s = sigma_rg_deg_s * deg2rad_val; % rad/s
52 sigma_rgb_deg_per_sqrt_Hz = 1e-3; % deg/s/sqrt(Hz)
53 sigma_rgb_rad_per_sqrt_Hz = sigma_rgb_deg_per_sqrt_Hz * deg2rad_val; %
    rad/s/sqrt(Hz)
54 %% Main Simulation Loop for Each Case (k=1 to 4)
55 results_L_applied = cell(1,4);
56 results_L_commanded = cell(1,4);
57 results_torque_diff_norm = cell(1,4);
58 results_euler_angles = cell(1,4);
59 results_angular_velocities = cell(1,4);
60 results_m_generated = cell(1,4);
61 for k_case = 1:4
62     fprintf('Running Simulation for Case k = %d\n', k_case);
63     omega_history = zeros(3, N_samples + 1);
64     euler_history = zeros(3, N_samples + 1);
65     dcm_history = zeros(3, 3, N_samples + 1);
66     L_applied_history = zeros(3, N_samples);
67     L_commanded_history = zeros(3, N_samples);
68     m_generated_history = zeros(3, N_samples);
69     omega_history(:, 1) = omega_initial;
70     euler_history(:, 1) = euler_initial;
71     dcm_history(:, :, 1) = euler_to_dcm(euler_initial(1),
        euler_initial(2), euler_initial(3));
72     b_rg_rad_s_current_sim = zeros(3, N_samples + 1);
73     b_rg_rad_s_current_sim(:, 1) = b_rg_0_rad_s;
74     for t_idx_bias = 1:N_samples
75         noise_rgb_step = sigma_rgb_rad_per_sqrt_Hz * randn(3,1) *
            sqrt(delta_t);
76         b_rg_rad_s_current_sim(:, t_idx_bias+1) =
            b_rg_rad_s_current_sim(:, t_idx_bias) + noise_rgb_step;
77     end
78     for i = 1:N_samples
79         omega_current = omega_history(:, i);
80         euler_current = euler_history(:, i);
81         C_BN_current = dcm_history(:, :, i);
82         t_sim = time_vec(i);
83         Bn_inertial_nT = [20000 * cos(2 * pi * t_sim / T_orbit);
84             20000 * sin(2 * pi * t_sim / T_orbit);
85             30000];
86         Bb_true_nT = C_BN_current * Bn_inertial_nT;
87         v_mag_nT = sigma_mag_nT * randn(3,1);
    
```

```

88     Bb_measured_case1_nT = Bb_true_nT + b_mag_nT + v_mag_nT;
89     Bb_measured_case2_nT = Bb_true_nT + v_mag_nT;
90     Bb_measured_case3_nT = Bb_true_nT;
91     omega_BN_true_current = omega_current;
92     v_rg_rad_s = sigma_rg_rad_s * randn(3,1);
93     b_rg_current_step_rad_s = b_rg_rad_s_current_sim(:,i);
94     omega_tilde_BN_case1_rad_s = omega_BN_true_current +
        b_rg_current_step_rad_s + v_rg_rad_s;
95     omega_tilde_BN_case2_rad_s = omega_BN_true_current + v_rg_rad_s;
96
97     B_b_for_control_nT = [];
98     omega_for_control_rad_s = [];
99     if k_case == 1
100         B_b_for_control_nT = Bb_measured_case1_nT;
101         omega_for_control_rad_s = omega_tilde_BN_case1_rad_s;
102     elseif k_case == 2
103         B_b_for_control_nT = Bb_measured_case1_nT;
104         omega_for_control_rad_s = omega_tilde_BN_case2_rad_s;
105     elseif k_case == 3
106         B_b_for_control_nT = Bb_measured_case2_nT;
107         omega_for_control_rad_s = omega_tilde_BN_case1_rad_s;
108     elseif k_case == 4
109         B_b_for_control_nT = Bb_measured_case2_nT;
110         omega_for_control_rad_s = omega_tilde_BN_case2_rad_s;
111     end
112     B_b_for_control_T = B_b_for_control_nT * 1e-9;
113     norm_B_b_for_control = norm(B_b_for_control_T);
114     if norm_B_b_for_control < 1e-9
115         b_hat_measured = [0;0;0];
116     else
117         b_hat_measured = B_b_for_control_T / norm_B_b_for_control;
118     end
119     if norm_B_b_for_control < 1e-9
120         m_gen_Am2 = [0;0;0];
121     else
122         m_gen_Am2 = -k_B / norm_B_b_for_control *
            cross(b_hat_measured, omega_for_control_rad_s);
123     end
124     m_generated_history(:,i) = m_gen_Am2;
125     Bb_true_T = Bb_measured_case3_nT * 1e-9;
126     L_C_applied_Nm = cross(m_gen_Am2, Bb_true_T);
127     L_applied_history(:,i) = L_C_applied_Nm;
128     b_hat_outer_b_hat = b_hat_measured * b_hat_measured';
129     L_C_commanded_Nm = -k_B * (eye(3) - b_hat_outer_b_hat) *
        omega_for_control_rad_s;
130     L_commanded_history(:,i) = L_C_commanded_Nm;
131     L_total_Nm = L_D + L_C_applied_Nm;

```

```

132     omega_dot = I_satellite \ (L_total_Nm - cross(omega_current,
133           I_satellite * omega_current));
134     omega_new = omega_current + omega_dot * delta_t;
135     omega_history(:, i+1) = omega_new;
136     psi_curr = euler_current(1);
137     theta_curr = euler_current(2);
138     phi_curr = euler_current(3);
139
140     if abs(cos(theta_curr)) < 1e-6
141         euler_dot = [0; omega_current(2); 0];
142         if mod(i,100)==0
143             warning('Singularity reached in Euler angles at t=%f
144               for case k=%d. Theta = %f deg.', t_sim, k_case,
145                 theta_curr*rad2deg_val);
146         end
147     else
148         Kinematic_Matrix = (1/cos(theta_curr)) * [ 0,
149             sin(phi_curr),          cos(phi_curr);
150             0,
151             cos(phi_curr)*cos(theta_curr),
152             -sin(phi_curr)*cos(theta_curr),
153             cos(theta_curr),
154             sin(phi_curr)*sin(theta_curr),
155             cos(phi_curr)*sin(theta_curr),
156             0];
157         euler_dot = Kinematic_Matrix * omega_current;
158     end
159     euler_new = euler_current + euler_dot * delta_t;
160     euler_history(:, i+1) = euler_new;
161     dcm_history(:, :, i+1) = euler_to_dcm(euler_new(1), euler_new(2),
162       euler_new(3));
163
164     end
165     results_L_applied{k_case} = L_applied_history;
166     results_L_commanded{k_case} = L_commanded_history;
167     results_torque_diff_norm{k_case} = vecnorm(L_commanded_history -
168       L_applied_history);
169     results_euler_angles{k_case} = euler_history;
170     results_angular_velocities{k_case} = omega_history;
171     results_m_generated{k_case} = m_generated_history;
172     fprintf('Finished Simulation for Case k = %d\n\n', k_case);
173
174 end
175
176 %% Text Output of Key Results
177 fprintf('\n\n--- NUMERICAL RESULTS SUMMARY ---\n');
178
179 output_step_interval = 500; % Ka rnekte bir veri
180 yazd r laca n belirler ( rnein 100, 200, 500). delta_t=1s
181 oldu u i in bu ayn zamanda saniye cinsinden aral kt r.

```

```

165 if output_step_interval <= 0; output_step_interval = 1; end % Hatal
    giri i engelle
166 time_points_for_output = unique([1,
    output_step_interval:output_step_interval:(N_samples+1),
    (N_samples+1)]);
167 for k_case_out = 1:4
168     fprintf('\n--- Results for Case k = %d ---\n', k_case_out);
169
170     current_L_applied = results_L_applied{k_case_out};
171     current_torque_diff_norm = results_torque_diff_norm{k_case_out};
172     current_euler_angles_rad = results_euler_angles{k_case_out};
173     current_angular_velocities_rad_s =
        results_angular_velocities{k_case_out};
174     fprintf('Initial Euler Angles (deg): [%.4f, %.4f, %.4f]\n',
        rad2deg_val*euler_initial(1), rad2deg_val*euler_initial(2),
        rad2deg_val*euler_initial(3));
175     fprintf('Initial Angular Velocities (deg/s): [%.4f, %.4f, %.4f]\n',
        rad2deg_val*omega_initial(1), rad2deg_val*omega_initial(2),
        rad2deg_val*omega_initial(3));
176
177     fprintf('\nTime-Stepped Data:\n');
178     fprintf('%-10s | %-25s | %-30s | %-30s | %-25s\n', 'Time(s)',
        'Euler Angles (deg)', 'Angular Velocities (deg/s)', 'Applied
        Control Torque (Nm)', 'Norm Torque Diff (Nm)');
179     fprintf('%s\n', repmat('-',1,130));
180     for tp_idx = 1:length(time_points_for_output)
181         t_step = time_points_for_output(tp_idx);
182         actual_time_s = (t_step-1)*delta_t;
183         if t_step == 1
184             actual_time_s = 0;
185         end
186
187         euler_rad_tp = current_euler_angles_rad(:, t_step);
188         omega_rad_s_tp = current_angular_velocities_rad_s(:, t_step);
189
190         torque_idx = min(t_step, N_samples);
191         if t_step == 1
192             L_app_tp = current_L_applied(:, torque_idx);
193             norm_diff_tp = current_torque_diff_norm(torque_idx);
194         else
195             torque_idx_eff = min(t_step-1, N_samples);
196             if torque_idx_eff == 0; torque_idx_eff = 1; end;
197             L_app_tp = current_L_applied(:, torque_idx_eff);
198             norm_diff_tp = current_torque_diff_norm(torque_idx_eff);
199         end
200
201         if actual_time_s == 0

```

```

202         fprintf('%-10.0f | [%.3f, %.3f, %.3f]          | [%.4f, %.4f,
           %.4f]          | (L_C at t=0 not stored)          | (Diff at
           t=0 not stored)\n', ...
203         actual_time_s, ...
204         rad2deg_val*euler_rad_tp(1),
           rad2deg_val*euler_rad_tp(2),
           rad2deg_val*euler_rad_tp(3), ...
205         rad2deg_val*omega_rad_s_tp(1),
           rad2deg_val*omega_rad_s_tp(2),
           rad2deg_val*omega_rad_s_tp(3));
206     else
207         fprintf('%-10.0f | [%.3f, %.3f, %.3f]          | [%.4f, %.4f,
           %.4f]          | [%.2e, %.2e, %.2e]          | %.2e\n', ...
208         actual_time_s, ...
209         rad2deg_val*euler_rad_tp(1),
           rad2deg_val*euler_rad_tp(2),
           rad2deg_val*euler_rad_tp(3), ...
210         rad2deg_val*omega_rad_s_tp(1),
           rad2deg_val*omega_rad_s_tp(2),
           rad2deg_val*omega_rad_s_tp(3), ...
211         L_app_tp(1), L_app_tp(2), L_app_tp(3), ...
212         norm_diff_tp);
213     end
214 end
215
216 fprintf('\nSummary Statistics (for t=1s to t=6000s for
           torques):\n');
217 mean_L_applied = mean(current_L_applied, 2);
218 std_L_applied = std(current_L_applied, 0, 2);
219 mean_torque_diff = mean(current_torque_diff_norm);
220 std_torque_diff = std(current_torque_diff_norm);
221
222 fprintf('  Mean L_applied (Nm): [%.2e, %.2e, %.2e]\n',
           mean_L_applied(1), mean_L_applied(2), mean_L_applied(3));
223 fprintf('  Std Dev L_applied (Nm): [%.2e, %.2e, %.2e]\n',
           std_L_applied(1), std_L_applied(2), std_L_applied(3));
224 fprintf('  Mean Torque Diff Norm (Nm): %.2e\n', mean_torque_diff);
225 fprintf('  Std Dev Torque Diff Norm (Nm): %.2e\n', std_torque_diff);
226 fprintf('%s\n', repmat('-',1,130));
227 end
228 %% Plotting Results ( devde istenen grafikler)
229 for k_case_plot = 1:4
230     time_plot_L = time_vec(1:N_samples);
231     time_plot_state = time_vec;
232     figure_name_L_app = sprintf('Applied Control Torques - Case %d',
           k_case_plot);
233     figure('Name', figure_name_L_app);

```

```

234 plot(time_plot_L, results_L_applied{k_case_plot}(1,:), 'r',
      'DisplayName', 'L_{applied,1}');
235 hold on;
236 plot(time_plot_L, results_L_applied{k_case_plot}(2,:), 'g',
      'DisplayName', 'L_{applied,2}');
237 plot(time_plot_L, results_L_applied{k_case_plot}(3,:), 'b',
      'DisplayName', 'L_{applied,3}');
238 hold off;
239 xlabel('Time (s)');
240 ylabel('Applied Control Torque (N.m)');
241 title(sprintf('Applied Control Torques (L_{applied}) - Case %d',
      k_case_plot));
242 legend show;
243 grid on;
244 figure_name_torque_diff = sprintf('Norm of Torque Difference - Case
      %d', k_case_plot);
245 figure('Name', figure_name_torque_diff);
246 plot(time_plot_L, results_torque_diff_norm{k_case_plot}, 'k');
247 xlabel('Time (s)');
248 ylabel('Norm of (L_{commanded} - L_{applied}) (N.m)');
249 title(sprintf('Norm of Torque Difference - Case %d', k_case_plot));
250 grid on;
251
252 euler_angles_plot_deg = rad2deg_val *
      results_euler_angles{k_case_plot};
253 euler_angles_plot_deg(1,:) = wrapTo180(euler_angles_plot_deg(1,:));
254 euler_angles_plot_deg(2,:) = wrapTo180(euler_angles_plot_deg(2,:));
255 euler_angles_plot_deg(3,:) = wrapTo180(euler_angles_plot_deg(3,:));
256 figure_name_euler = sprintf('Euler Angles - Case %d', k_case_plot);
257 figure('Name', figure_name_euler);
258 plot(time_plot_state, euler_angles_plot_deg(1,:), 'r',
      'DisplayName', '\psi (Yaw)');
259 hold on;
260 plot(time_plot_state, euler_angles_plot_deg(2,:), 'g',
      'DisplayName', '\theta (Pitch)');
261 plot(time_plot_state, euler_angles_plot_deg(3,:), 'b',
      'DisplayName', '\phi (Roll)');
262 hold off;
263 xlabel('Time (s)');
264 ylabel('Euler Angles (deg)');
265 title(sprintf('Euler Angles - Case %d', k_case_plot));
266 legend show;
267 grid on;
268 ylim([-180 180]);
269 angular_velocities_plot_deg_s = rad2deg_val *
      results_angular_velocities{k_case_plot};

```

```

270     figure_name_omega = sprintf('Angular Velocities - Case %d',
        k_case_plot);
271     figure('Name', figure_name_omega);
272     plot(time_plot_state, angular_velocities_plot_deg_s(1,:), 'r',
        'DisplayName', '\omega_1');
273     hold on;
274     plot(time_plot_state, angular_velocities_plot_deg_s(2,:), 'g',
        'DisplayName', '\omega_2');
275     plot(time_plot_state, angular_velocities_plot_deg_s(3,:), 'b',
        'DisplayName', '\omega_3');
276     hold off;
277     xlabel('Time (s)');
278     ylabel('Angular Velocity (deg/s)');
279     title(sprintf('Angular Velocities - Case %d', k_case_plot));
280     legend show;
281     grid on;
282 end
283 disp('Simulations and plotting complete.');
```

%% Helper Functions

```

285 function C_BN = euler_to_dcm(psi, theta, phi)
286     C_psi = [cos(psi) sin(psi) 0; -sin(psi) cos(psi) 0; 0 0 1];
287     C_theta = [cos(theta) 0 sin(theta); 0 1 0; -sin(theta) 0
        cos(theta)]; % Standart Ry(theta)
288     C_phi = [1 0 0; 0 cos(phi) sin(phi); 0 -sin(phi) cos(phi)];
289     C_BN = C_phi * C_theta * C_psi;
290 end

```

Listing 1: Main MATLAB Simulation Script
(HW5_ADCS_Magnetorquers_Simulation.m)